UNIDAD ACADÉMICA MULTIDISCIPLINARIA MANTE

DISEÑO ELECTRÓNICO BASADO EN SISTEMAS EMBEBIDOS

Proyecto Integrador 1. Sistema de Alerta por Temperatura y Humedad Ambiente

Alumnos(as):

- Castro Márquez Cristian
- García Segura América Yanely
- Hernández Ruiz Haydee Michelle
- Montoya Garza Luis Ángel
- Navarro García Maribel
- Rueda Martínez Alison Michelle
- Silva Sánchez Yamilka Arely

8° “E”

Maestro(a):

Dr. López Piña Daniel

Ciudad Mante, Tamaulipas Abril, 2026

Contenido

Rol de cada integrante 3
- **Estudiantes 1 y 2 – Lista de Materiales** 3
- **Estudiante 3 – Diseñador de Circuito** 3
- **Estudiante 4 – Montador de Hardware** 3
- **Estudiante 5 – Programador** 3
- **Estudiante 6 – Tester / QA** 3
- **Estudiante 7 – Documentador del Proceso** 3

Objetivo 3
- **Etapa 1** 4
  - **Estudiantes 1 y 2 – Análisis de Requisitos** 4
  - **Estudiantes 1 y 2 – Lista de Materiales** 4
  - **Estudiantes 1 y 2 – Mapa de Pines** 6
- **Etapa 2** 7
  - **Estudiante 3 – Diseñador de Circuito** 7
  - **Estudiante 3 – Tabla de conexiones: componente → pin Arduino → función** 8
- **Etapa 3** 9
  - **Estudiante 4 – Montador de Hardware** 9
- **Etapa 4** 10
  - **Estudiante 5 – Programador** 10
  - **Estudiante 5 – Descripción de la máquina de estados implementada** 19
- **Etapa 5** 20
  - **Estudiante 6 – Tester / QA** 20
- **Etapa 6** 22
  - **Estudiante 7 – Documentador del Proceso** 22

Conclusión 22

Rol de cada integrante

Estudiantes 1 y 2 – Lista de Materiales

Navarro García Maribel

Silva Sánchez Yamilka Arely

Estudiante 3 – Diseñador de Circuito

Hernández Ruiz Haydee Michelle

Estudiante 4 – Montador de Hardware

Montoya Garza Luis Ángel

Estudiante 5 – Programador

Rueda Martínez Alison Michelle

Estudiante 6 – Tester / QA

Castro Márquez Cristian

Estudiante 7 – Documentador del Proceso

García Segura América Yanely

Objetivo

Desarrollar un sistema embebido que permita monitorear la temperatura y humedad usando el sensor DHT22, mostrando los datos en una pantalla OLED SSD1306 mediante la comunicación I2C. El sistema enviará una alerta visual con un LED y una sonora usando un buzzer cuando los valores superen los límites establecidos mediante pulsadores. Los datos se van a registrar en un Monitor Serial en tiempo real.

Etapa 1

Estudiantes 1 y 2 – Análisis de Requisitos

REQ-01. El sistema debe leer temperatura y humedad cada 2 segundos usando DHT22

REQ-02. La pantalla OLED debe mostrar: temperatura (°C), humedad (%HR), estado del sistema y hora de arranque

REQ-03. Umbral de temperatura configurable con pulsadores (botón + sube 1°C, botón – baja 1°C). Rango: 20°C-40°C.

REQ-04. LED verde = condición normal; LED amarillo = advertencia (±2°C del umbral); LED rojo = alarma superado umbral

REQ-05. El buzzer emite 3 pitidos cortos cuando se activa la alarma y 1 pitido largo al retornar al estado normal

REQ-06. El Monitor Serial envía datos cada lectura con formato: [t=Xms] Temp: X.X°C | Hum: X.X% | Estado: NORMAL/ALERTA

REQ-07. El sistema no debe bloquearse (uso de millis() en lugar de delay())

Estudiantes 1 y 2 – Lista de Materiales

- Arduino UNO o Nano - 1 unidad
- Sensor DHT22 - 1 unidad
- Pantalla OLED SSD1306 (I2C, 128x64) - 1 unidad
- LED RGB (o 3 LEDs: rojo, amarillo, verde) - 1 unidad / 3 unidades
- Buzzer piezoeléctrico pasivo - 1 unidad
- Pulsadores (push buttons) - 2 unidades
- Resistencias 220 Ω - 3 unidades
- Resistencias 10 kΩ - 2 unidades

- Protoboard - 1 unidad
- Cables dupont M-M y M-H - 30 unidades aprox.
- Cable USB A-B - 1 unidad

Imagen 1. Lista de Materiales

Estudiantes 1 y 2 – Mapa de Pines

VCC(positivo +)

GND(negativo -)

Componente	Pines al Arduino
Sensor dht22	
Out	2
VCC	positivo
GND	negativo
Pantalla OLED	
GND	negativo
VCC	positivo
SCL	A5
SDA	A4
LED (4 pines)	
R	9
GND	negativo
G	10
B	11
Buzzer	
GND	negativo
VCC	8
Pulsador 1	
GND	negativo
El otro pin	4
Pulsador 2	
GND	negativo
El otro pin	5

NOTA:

- Puentear el GND del Arduino (negativo) a la protoboard.
- Sacar 5v del Arduino al (positivo) de la protoboard.

Etapa 2

Estudiante 3 – Diseñador de Circuito

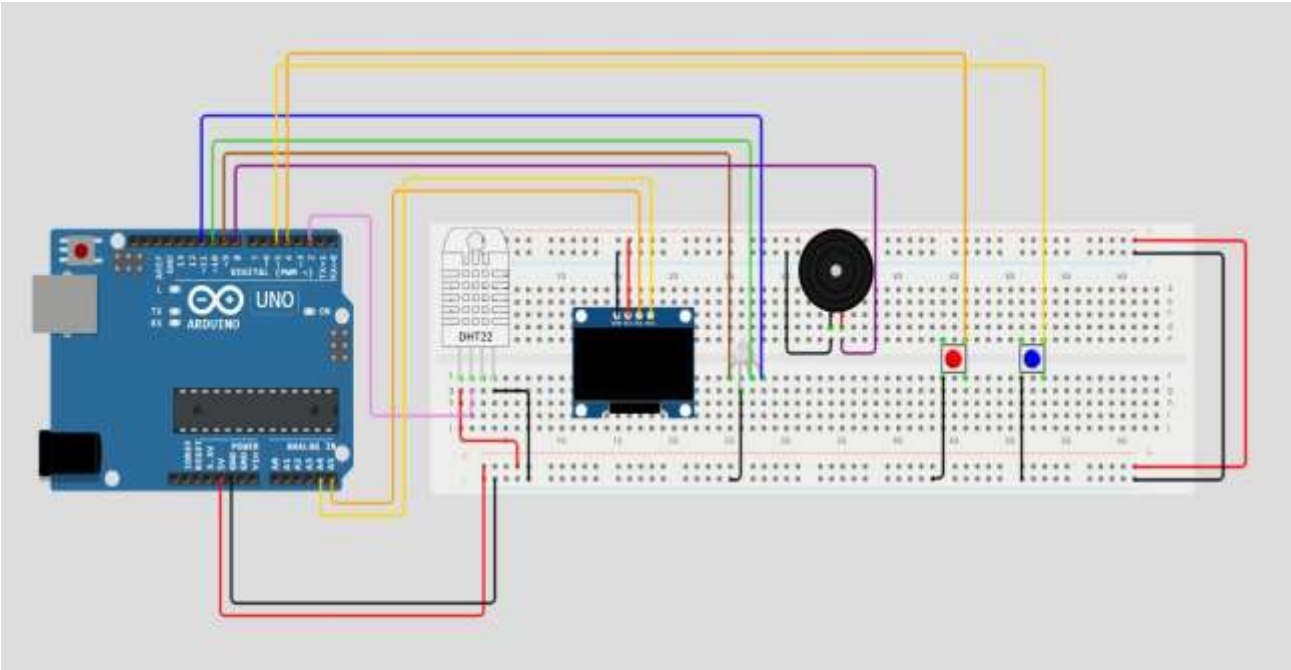

Imagen 2. Esquema de conexión en Wokwi

Estudiante 3 – Tabla de conexiones: componente → pin Arduino → función

Componente	Pines al Arduino	Función
Sensor DHT22		
Out	2	Envía datos digitales al arduino.
VCC	Positivo	Alimenta el sensor
GND	Negativo	Cierra el circuito
Pantalla Oled		
GND	Negativo	Tierra del circuito
VCC	Positivo	Alimenta la pantalla
SCL	A5	Señal de reloj que sincroniza
SDA	A4	Transmite los datos de texto o números a la pantalla
LED RGB		
R	9	Estado de alarma cuando supera el umbral
GND	Negativo	Tierra del circuito
G	10	Estado de advertencia
B	11	Estado normal
Buzzer		
GND	Negativo	Tierra del circuito
VCC	8	Emite sonido cuando se activa la alarma
Pulsador 1		
GND	Negativo	Tierra del circuito
Otro pin	4	Aumenta el umbral
Pulsador 2		
GND	Negativo	Tierra del circuito
Otro pin	5	Disminuye el umbral

Etapa 3

Estudiante 4 – Montador de Hardware

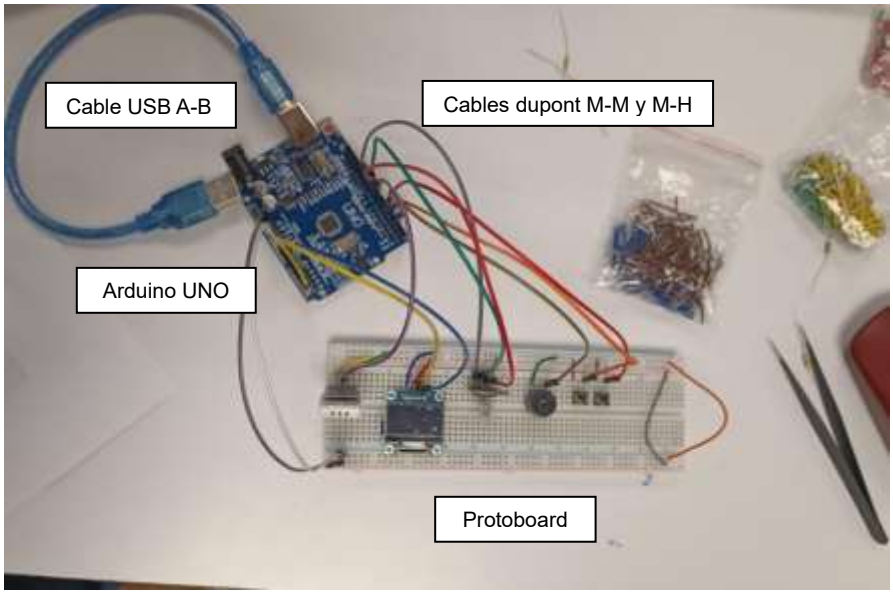

Imagen 3. Circuito físico montado en protoboard

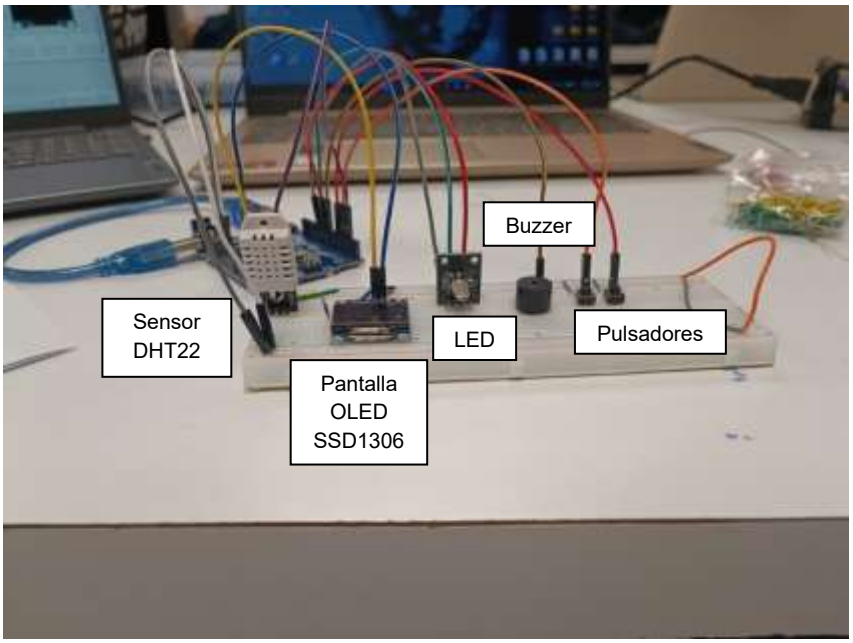

Imagen 4. Circuito físico montado en protoboard

Etapa 4

Estudiante 5 – Programador

```
// ==============================================================
// PROYECTO INTEGRADOR 1 — Sistema de Alerta Temp & Humedad
// LED RGB cátodo común
// ==============================================================

//Librerias
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

// ── Inicializar la OLED
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// ── inicializar el DHT22 con la libreria y definir los pines
#define DHTPIN  2
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

// ── PINES
//LED RGB
```

```
#define RGB_R    9
#define RGB_G    10
#define RGB_B    11
//BUZZER
#define BUZZER   8
//BOTONES
#define BTN_SUBE 4
#define BTN_BAJA 5

// ── ESTADOS
#define NORMAL      0
#define ADVERTENCIA 1
#define ALARMA      2

// ── VARIABLES
float temperatura = 0.0;
float humedad     = 0.0;
int   umbralTemp  = 30;
int   estado      = NORMAL;
int   estadoPrev  = -1;

unsigned long tLectura  = 0;
unsigned long tBuzzer   = 0;
unsigned long tArranque = 0;
unsigned long tDebBtn   = 0;
```

```
bool buzzerON    = false;
int  buzzerPaso  = 0;
int  buzzerTotal = 0;
bool buzzerLargo = false;

// ── RGB helper
void setRGB(bool r, bool g, bool b) {
  digitalWrite(RGB_R, r);
  digitalWrite(RGB_G, g);
  digitalWrite(RGB_B, b);
}

//
──────────────
```

```
  setRGB(0,0,0);
  dht.begin();
  // comprobar que funciona el oled
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("Error OLED"); while(true);
  }
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0,0);
  display.println("Iniciando...");
  display.display();
  tArranque = millis();
  delay(1000);
}

// LOOP PRINCIPAL
void loop() {
  unsigned long ahora = millis();
  leerBotones(ahora);
  leerSensores(ahora);
  actualizarEstado();
  actualizarLED();
  gestionarBuzzer(ahora);
  actualizarOLED(ahora);
}
```

```
// ── BOTONES debounce 200 ms
void leerBotones(unsigned long ahora) {
  if (ahora - tDebBtn < 200) return;
  if (digitalRead(BTN_SUBE) == LOW) { if (umbralTemp < 40) umbralTemp++;
tDebBtn = ahora; }
  if (digitalRead(BTN_BAJA) == LOW) { if (umbralTemp > 20) umbralTemp--;
tDebBtn = ahora; }
}

// ── LECTURA cada 2 s
void leerSensores(unsigned long ahora) {
  if (ahora - tLectura < 2000) return;
  tLectura = ahora;
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  if (!isnan(t)) temperatura = t;
  if (!isnan(h)) humedad     = h;

  String est = (estado == NORMAL) ? "NORMAL" :
               (estado == ADVERTENCIA) ? "ADVERTENCIA" : "ALERTA";
  Serial.print("[t="); Serial.print(ahora);
  Serial.print("ms] Temp: "); Serial.print(temperatura,1);
  Serial.print("C | Hum: "); Serial.print(humedad,1);
  Serial.print("% | Estado: "); Serial.println(est);
}
```

```
// ── MÁQUINA DE ESTADOS
//Umbral: 30°c
//ALARMA: La temperatura es mayor que el umbral. Prende el LED de color rojo y el buzzer da 3 pitidos
//ADVERTENCIA: La temperatura está 2°c mas bajo del umbral. El LED prende de color amarillo, sin sonido de buzzer
//NORMAL: La temperatura es menor que el umbral. El LED prende de color Verde. sin sonido de buzzer
void actualizarEstado() {
  int nuevo;
  if      (temperatura >= umbralTemp)     nuevo = ALARMA;
  else if (temperatura >= umbralTemp - 2) nuevo = ADVERTENCIA;
  else                                    nuevo = NORMAL;

  if (nuevo != estadoPrev) {
    //Transicion de ALARMA a NORMAL: 1 pitido largo del buzzer
    if (nuevo == ALARMA)                          iniciarBuzzer(3, false);
    else if (nuevo == NORMAL && estadoPrev == ALARMA) iniciarBuzzer(1, true);
    estadoPrev = nuevo;
  }
  estado = nuevo;
}

// ── LED RGB
void actualizarLED() {
  switch(estado) {
```

```
    case NORMAL:      setRGB(0,1,0); break;  // verde
    case ADVERTENCIA: setRGB(1,1,0); break;  // amarillo
    case ALARMA:      setRGB(1,0,0); break;  // rojo
  }
}

// ── BUZZER no bloqueante
void iniciarBuzzer(int pulsos, bool largo) {
  buzzerTotal = pulsos * 2;
  buzzerPaso  = 0;
  buzzerLargo = largo;
  buzzerON    = true;
  tBuzzer     = millis() - 9999;
}

void gestionarBuzzer(unsigned long ahora) {
  if (!buzzerON) return;
  unsigned long durON  = buzzerLargo ? 600 : 150;
  unsigned long durOFF = buzzerLargo ? 200 : 150;
  unsigned long dur    = (buzzerPaso % 2 == 0) ? durON : durOFF;
  if (ahora - tBuzzer >= dur) {
    tBuzzer = ahora;
    if (buzzerPaso % 2 == 0) tone(BUZZER, buzzerLargo ? 880 : 1400);
    else                     noTone(BUZZER);
    buzzerPaso++;
    if (buzzerPaso >= buzzerTotal) { noTone(BUZZER); buzzerON = false; }
```

```
  }
}

// ── OLED
void actualizarOLED(unsigned long ahora) {
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);

  display.setTextSize(1);
  display.setCursor(20, 0);
  display.println("TEMP & HUMEDAD");
  display.drawLine(0, 9, 127, 9, SSD1306_WHITE);

  display.setTextSize(2);
  display.setCursor(0, 13);
  display.print(temperatura, 1);
  display.println("C");

  display.setTextSize(1);
  display.setCursor(0, 33);
  display.print("Hum: "); display.print(humedad,1); display.println("%");

  display.setCursor(0, 43);
  display.print("Umbral: "); display.print(umbralTemp); display.println("C");

  display.setCursor(0, 53);
```

```
  display.print("Estado: ");
  switch(estado) {
    case NORMAL:      display.print("NORMAL");      break;
    case ADVERTENCIA: display.print("ADVERTENCIA"); break;
    case ALARMA:      display.print("ALERTA!");     break;
  }
  display.display();
}
```

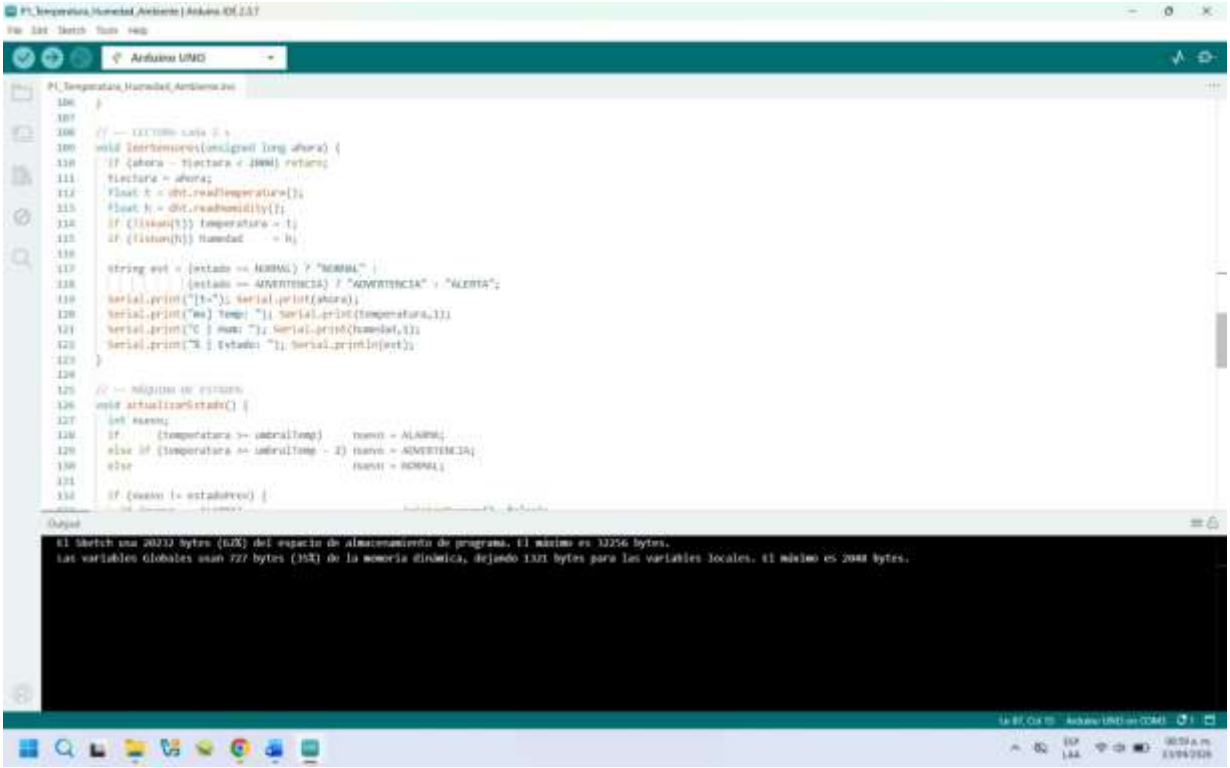

Imagen 5. El código compila sin errores

Estudiante 5 – Descripción de la máquina de estados implementada

Estado	Condición de entrada	Acciones
NORMAL	temp < umbral – 2°C	LED verde, sin buzzer
ADVERTENCIA	temp dentro de +-2°C del umbral	LED amarillo
ALARMA	temp > umbral	LED rojo, 3 pitidos cortos

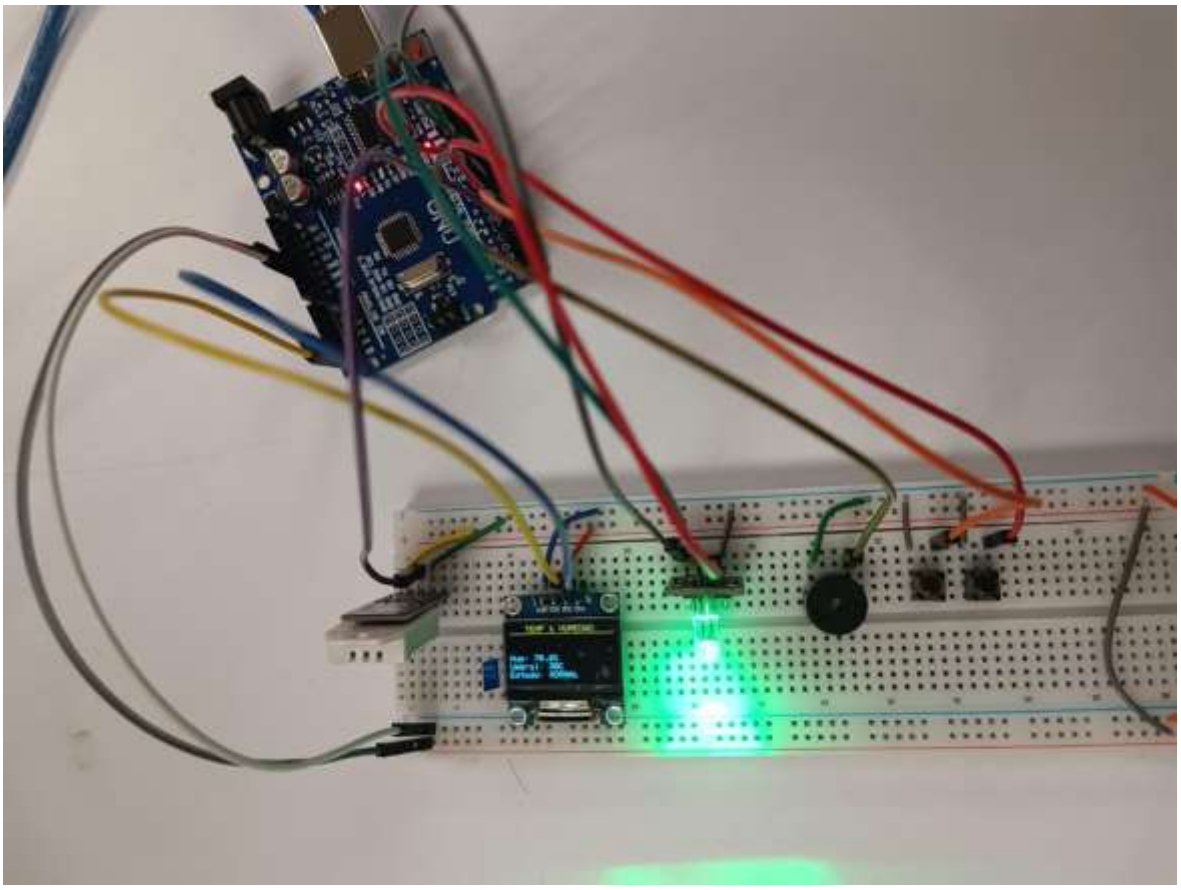

Imagen 6. Código cargado en el circuito

Etapa 5

Estudiante 6 – Tester / QA

REQUERIMIENTOS	RESULTADOS
REQ-01 Verificar que la lectura se realiza cada ~2 segundos	*PASS*
REQ-02 Verificar información en OLED	*PASS*
REQ-03 Presionar botones y verificar que el umbral cambia	*PASS*
REQ-04 Verificar cambio de color del LED según la temperatura	*PASS*
REQ-05 Verificar tonos del buzzer en alarma y retorno	*FAIL*
REQ-06 Verificar formato del monitor serial	PASS
REQ-07 Verificar que el sistema no se bloquea	PASS

OBSERVACIONES:

Todos los requerimientos han sido probados y la mayoría cumple con lo estipulado, a excepción del Requerimiento REQ-05, el cual a continuación se describen sus observaciones.

REQ-05:

Al momento de presionar el botón para disminuir el umbral el sistema si emite los 3 pitidos cortos, pero cuando se presiona el botón para aumentar el umbral y retornarlo al estado normal el sistema no emite el pitido largo.

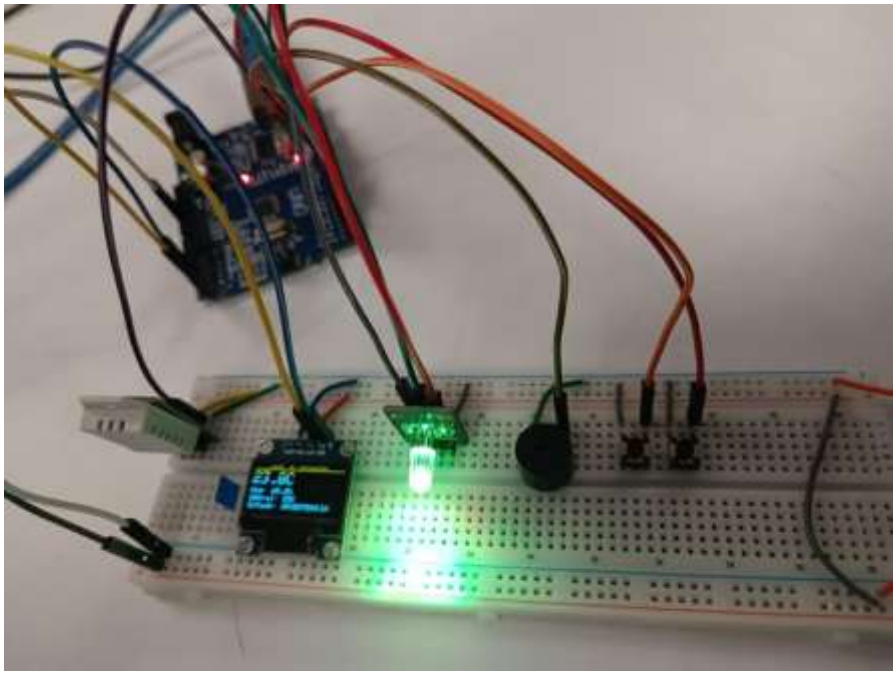

Imagen 7. Cambio de color del LED según temperatura

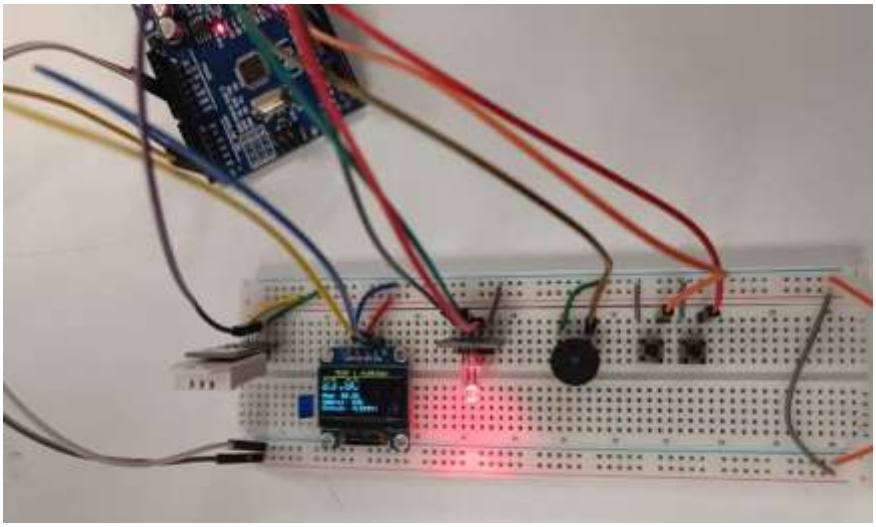

Imagen 8. Cambio de color del LED según temperatura

Etapa 6

Estudiante 7 – Documentador del Proceso

Conclusión

Se realizó un sistema que permite medir la temperatura y la humedad, mostrando los datos en una pantalla y activando alertas cuando se superan los límites establecidos. Este proyecto permite monitorear las condiciones del ambiente en tiempo real y reaccionar ante cambios importantes. Únicamente se detectó un error en el REQ-05, donde no emite un pitido largo al retornar al estado normal.